

REG M-C — the ledger

Version: 1.0.0 — 2026-09-24. Line: abra/regmc — `CHANGELOG-REGMC.md` .

A leading `0` means NOT USABLE YET (SemVer 2.0.0 clause 4). This line reaches **1.0.0 the day the M-C gate opens**, which is the same condition MEDICHAM met for Reg M-B: the differential at zero on every lattice and on the held-out draw, the roster and staged battery clean, and no open defect an instrument measures. Nothing is typed to declare it — `engine/quarantine.js` computes it.

The M-C gate is `node engine/quarantine.js --regulation regmc (0.16.0)`. It reads only `data/<name>-regmc.<ext>` artifacts and the `data/team-pool-frozen-regmc` pool, over its own lattice. A missing artifact reads NO ARTIFACT or CANNOT-ANSWER, never Reg M-B's file.

STATUS AT 1.0.0 (2026-09-24): THE GATE IS OPEN, 10 OF 10. Release `ea5becc54eb` , census pin `123aa264f88d` (1024 live / 0 missing, `data/mechanics-census-regmc.json:live`), pool `data/team-pool-frozen-regmc` , lattices `--games` 1200/1600/1900. Boards part in 0/955, 0/1266, 0/1497 games; undeclared commentary-only differences are 0/955, 0/1266, 0/1497, and the narration bar is stamped at zero (`data/whole-game-baseline-regmc.json`). Damage 0/6000 at every roll index; roster 166/166 items, 210/214 abilities, 510/511 moves; 4,867 staged games, 0 threw; no open register row names a RED instrument. Every Reg M-C artifact downstream of MEDICHAM is now RE-RUNNABLE, not current: none may be quoted until it is re-run on this release or later. Account: `docs/_reports/2026-09-24-regmc-gate-final.md` .

Reg M-B's published record is **7.0.0** and is closed. Nothing here changes it.

THE VERSION SCHEME — WHAT A NUMBER MEANS, WHEN IT RESETS, WHAT IS IMMUTABLE

WILL, 2026-09-20: *"i dont want high abra numbers we have just been working on medicham"*, against his standing brief *"i just want low numbers for when projects are actually done and usable"*; and then, on being shown the per-regulation scheme, *"much better and lets do it per model inside of abra if that makes sense"*.

A VERSION ANSWERS ONE QUESTION: HOW DONE IS THIS THING, ON THIS REGULATION. Nothing else. `7.0.0` did not mean that — it counted measurements, climbing at a rate of several releases a day whether or not anything became usable. (The release cadence that made that true is measured and argued in `engine/docs_scan.js` , at `OWED_CAP` ; it is not restated here.)

The key is a pair

A LINE	a version series for one (<i>project or model, regulation</i>) pair. It has its own changelog, its own document floor, its own notes rows and its own backlog.
<code>0.x</code>	NOT USABLE YET — SemVer 2.0.0 clause 4.
<code>1.0.0</code>	usable: a gate certifies it on that regulation, and nothing it publishes is withheld.
RESET	a new regulation starts a NEW line at <code>0.1.0</code> . It never inherits the old line's number.
IMMUTABLE	a closed line. Nothing in it is renumbered, rewritten or restated, ever.

Nothing above is typed into a list. A line is discovered by its changelog existing and declaring itself in its own masthead; a document declares its line in its masthead beside the version the line scopes:

```
node engine/docs_scan.js --lines      # the series that exist, which is open, each one's floor
node engine/model_versions.js         # the per-(model, regulation) version, derived from the gate
```

Where a model has a gate, its 1.0.0 is COMPUTED, never typed

`engine/model_versions.js` gives a model `1.0.0` on a regulation only when **all** of these hold, and prints which clause is failing when they do not:

- a gate exists that certifies that model on that regulation — MEDICHAM's is `engine/quarantine.js`, and the binding is derived from the gate's own `SIMULATOR` constant rather than typed;
- that gate is **OPEN**;
- the model publishes at least one artifact, and **not one of them is quarantined or stale**.

A MODEL WITH NO GATE CANNOT LEAVE `0.x`, AND THAT IS THE ANSWER RATHER THAN A GAP. MAG, MILTANK, PORYGON2, DODUO and GARY each decide their question by playing games on MEDICHAM; every figure they had was deleted at 7.0.0 because the engine beneath it changed. No bar is invented to make a number available for them. SLOWKING's team-preview half is the one declared exception and it is declared in `docs/MODELS.md`, not here: it solves a matrix built from real ladder outcomes with no simulator in its path, so it is upstream of MEDICHAM rather than downstream.

THE PROPERTY THAT MATTERS MOST: a model whose figures were measured under a superseded engine **cannot read 1.0.0**. That is why the third clause reads the quarantine and the provenance rather than anyone's judgement. One shared version is exactly why a page of stale MAG figures sat beside fresh MEDICHAM ones in `docs/MODELS.md` and read as equally authoritative — one of them disagreeing with `data/policy-weights.json` on disk as well as with the engine. They had to be deleted by hand at 7.0.0. A per-model version puts that in the number instead of relying on somebody noticing.

What is refused, so that "closed" cannot decay into a habit

- A changelog entry or a notes row above the `closed=` version its line declares — `tests/test-docs-current.js`.
- A document whose masthead names a line no changelog declares.
- A `0.x` line whose documents trail its TOP. A line with no major has no deferred pass, so its documents are due **every release**; that is stricter than the Reg M-B rule and it relaxes by itself the day the line reaches 1.0.0.
- Everything the gates refused before this change, unchanged: a basis change released as anything but `x.0.0`, an `x.0.0` naming no basis change, a PATCH that supersedes a figure, a document trailing its major floor, a figure a cited artifact does not contain.

One thing is still owed

The entry published as `7.2.0` (Will's three Reg M-C decisions — the M-B collector off, the scope, the Eject Button cut) was uncommitted when this landed and could not be moved from a worktree. It belongs to this line as `0.3.0`, content unchanged. The closed-line clause will name it by version and file until it is moved; that is the refusal working, not a regression.

THE TWO CHECKOUTS, AND WHY THERE ARE TWO

	path	HEAD	role
M-B authority	C:/Users/willj/Projects/Pokemon/pokemon-showdown	20ad99f , 2026-07-22	PINNED. Never pull it, never build it, never modify it. Every

	path	HEAD	role
			figure in ABRA 7.0.0 rests on these bytes.
M-C authority	C:/Users/willj/Projects/Pokemon/pokemon-showdown-mc	f10d679 , 2026-09-20	The moving one. M-C work reads this.

The separation is the whole point. A single checkout that gets pulled would move the authority under M-B's published figures — the same failure as a measurement reading a live store, one level up.

M-B DID NOT DRIFT, AND THIS WAS MEASURED RATHER THAN ASSUMED. The legal species set of `gen9championsvgc2026regmb` is identical in both checkouts — 347 either way, 0 added, 0 removed (CHANGELOG-REGMC 0.1.0). So adding the second checkout disturbed nothing that 7.0.0 published.

AND CHOOSING BETWEEN THEM IS NOW A FLAG, NOT AN EDIT — 0.10.0, 2026-09-21.

Which checkout loads is part of which REGULATION a run is about, and both are stated in one place:

```
node <anything>                # regmb - data/regulations.json `active`, the default
node <anything> --regulation regmc  # regmc - and it brings pokemon-showdown-mc with it
ABRA_REGULATION=regmc node <anything>  # same, for a script that refuses unknown flags
```

`engine/regulation.js` resolves it and `champions_sim.FORMAT` reads it, so every caller follows with no edit. `data/regulations.json` carries a `runtime` block naming each regulation's checkout and pinned commit, and `engine/showdown_path.js` tries the selected regulation's checkout first — an explicit `SHOWDOWN_PATH` still wins, for a checkout kept somewhere else. **Any deviation from the default prints itself on stderr**; an unresolvable `--regulation` REFUSES rather than falling back.

`regmc` **lives in** `runtime` **and deliberately NOT in** `regulations`. `engine/next_regulation.js` walks the `regulations` map to decide what is already known, so an entry there would tell the hourly collector that M-C is known and stop it collecting. **Being selectable is not being active** — `active` is still `regmb`.

THE FLAG IS NECESSARY AND IT IS NOT SUFFICIENT. Selecting M-C gets the right authority at the right format and a runtime that cannot field it: `data/engine-data.js` has no row for any of the added M-C species or moves, so `buildMon` returns null for all of them (`docs/_reports/2026-09-20-regmc-first-run.md` §3). That is a refit item. Counts, the before/after proof that Reg M-B is unmoved, and what remains: `CHANGELOG-REGMC.md` 0.10.0 and `docs/_reports/2026-09-21-regulation-runtime.md`.

THE DELTA, DERIVED AGAINST THE PINNED AUTHORITY

Full account, with every list and every derivation command: [docs/_reports/2026-09-20-regmc-delta.md](#).

These counts are a LIVE DERIVATION over the two checkouts, not a row in a `data/*.json` artifact, so they carry no artifact citation and are quoted as the readout that produced them:

```
$ node -e "compare Dex.forFormat('...regmb') in the PINNED checkout
          against Dex.forFormat('...regmc') in the M-C checkout"

species    +35    -0
moves      +15    -0
items      +18    -0
abilities   +0     -0
ruleset     identical: same expanded rules, value rules, banlists, timer

new to the format, by kind:  abilities 15    moves 14    held items 12    = 41 mechanics
added species bringing nothing new:  13
```

M-C is a strict superset. Nothing is removed anywhere.

THE REAL SURFACE IS THE MECHANIC COUNT, NOT THE SPECIES COUNT — a third of the added species bring nothing new at all, so counting species would have overstated the work and pointed it at the wrong things.

The changes that no legality list reveals

Two moves are legal in BOTH regulations and had their PP cut. Verified directly against both checkouts:

```
$ node -e "MB(pinned).moves.get(id).pp  vs  MC.moves.get(id).pp"

wish          pp  10 -> 5
strengthsap    pp  10 -> 5
```

A changed number on an entity that exists in both is the dangerous kind of change: every "what was added / what was removed" list shows nothing, and an engine built by assuming *M-B plus the new things* carries the old value forever. One species also loses two moves and gains one, confirmed in both directions by the validator.

Rocky Helmet is UNBANNED in M-C — `isNonstandard` moves from "Past" to `null`. The umbrella `CLAUDE.md` ban list is correct for M-B and becomes wrong for M-C the day this line goes active. The other six banned items and all three banned moves are unchanged. The engine models it from 0.18.0 (item tag `punishesAttackerItem`, `tests/probe_regmc_rocky_helmet.js`); before that it was tagged `flingable` only and did nothing. Air Balloon, back from `Past` the same way, is modelled from 0.20.0 (`poppedOnHit`, `tests/probe_regmc_air_balloon.js`); Red Card and Eject Button from 0.21.0 (`dragsAttackerOnHit`, `ejectsHolderOnHit`, `tests/probe_regmc_eject_items.js`). Emergency Exit, whose Champions override also keeps a pivot's switch, from 0.22.0 (`switchesOutAtHalf`, `tests/probe_regmc_emergency_exit.js`).

A pivot move plus Eject Button behaves differently. Showdown commit `aa6d5f0856` (2026-09-13, "*Champions: Allow self-switches even if Eject Button is triggered*") adds an Eject Button override that is the mainline handler **minus** the line clearing the attacker's switch flag: both bodies now switch out. The engine follows it from 0.21.0 and reads the rule off the handler (`ejectsHolderOnHit.cancelsSourceSwitch`), so the checkout that is loaded decides it. **That fix landed four days AFTER M-C went live on 2026-09-09**, so M-C replays collected in that window were played under the old rule. Any measurement over the M-C store must either exclude that window or account for it.

THE TRAP THAT WILL BITE FIRST

THE STRICT LEGALITY FILTER SILENTLY DELETES A LIVE ABILITY.

CLAUDE.md requires every dex walk to be filtered — `x.exists && !x.isNonstandard && x.tier !== 'Illegal'` — and that rule is right, and was written because an unfiltered walk once invented five exceptions to a rule that has none. **In M-C it drops one entity that is real.**

One ability introduced with the new mega formes carries `isNonstandard: "Future"`, which the filter rejects. But it is **live**: the validator accepts a legal set carrying it, and it has a real handler body that halves contact damage. Measured on the M-C checkout, as a quoted readout:

```
$ node -e "abilities under gen9championsvgc2026regmc"
passing the strict filter          316
flagged Future and dropped by it   1  <- live, validator-accepted, has a handler body
```

So a generator, a census walk or a roster stage built on the strict filter will never see it, will never stage it, and will report full coverage while doing so — **a capability absent, with everything reporting success**, which is this project's signature failure mode.

The rule for M-C: filter every walk as before, then **re-admit the** `Future` **entries the validator accepts**, and print the re-admitted list on every run so it can never be silently empty. `"Future"` is also what every Champions-exclusive entity carries in mainline data — including ones already modelled for M-B — so the flag alone means nothing either way. The question is always whether a handler body exists and whether the validator accepts a set carrying it.

WHAT REG M-B LEFT OPEN, DEFERRED BY WILL RATHER THAN CLOSED

Will, 2026-09-20, asked whether MEDICHAM was done for Reg M-B and, shown the two items below, answered **"move to reg mc"**. They are DEFERRED, not finished, and they are written here rather than only in a dated row so that the next session reads them without having to go looking.

- **7 live** `lastMove` **readings disagree**. The authority keeps the CALLING move where this engine keeps the called one. Found 2026-09-20 in the held-out dump, reported, **not fixed**. It is a real unregistered class: no instrument gates it, which is exactly why it did not block the gate opening.
- **The corner arms were never read on the final release**. The extreme-damage-roll arms were last run on two superseded releases and have never been read on `0d7b1d9db6d1`, so no corner figure describes the engine ABRA published at 7.0.0.

Neither is a Reg M-C item and neither is closed. If a Reg M-C measurement ever appears to disagree with a published Reg M-B figure, check these two first: the corner arms in particular are the obvious candidate for a difference that looks like a regulation change and is not.

THE THREE DECISIONS WILL TOOK, 2026-09-20

1. THE REG M-B COLLECTOR IS OFF. *"we can turn off the reg mb collector we have moved"*.

`.github/workflows/ingest.yml` has its schedule commented out; `workflow_dispatch` is kept so a catch-up pull is one click away, because that store is the frozen authority behind `data/team-pool-frozen` and every figure ABRA published at 7.0.0. It would have gone quiet anyway: the M-B format carries `searchShow: false` in the M-C checkout, so it cannot be laddered and its rolling replay pool has stopped refilling. Reg M-C's collector is a different workflow and is untouched.

2. THE SCOPE IS THE SAME AS REG M-B, FOR NOW. *"yes lets do the same scope as mb for now".* Open team sheets only; Illusion remains the one declared exclusion; closed sheets and bo1 stay out of scope. This is a decision to revisit once the format is understood, not a permanent boundary — the added mechanics may change what is worth modelling.

3. ONLY THE GAMES THE RULE COULD HAVE TOUCHED COME OUT. Will: *"sure remove the old eject button rule game"*, and then, when the first cut was by DATE: *"i mean can we just remove the games that have an eject button on the team sheet"*.

The exclusion is a CONJUNCTION, not a date range: a game leaves the pool only if it was played before the fix AND an Eject Button is declared on a team sheet. The rule change cannot have altered a game in which the item never appears, so excluding such a game discards evidence for nothing.

```
$ count bo3 games by (date < 2026-09-14) x (Eject Button on either sheet)    (2026-09-20)
total                                                                    23808
played before the fix                                                    8352    (35.1%)
Eject Button on a sheet, any date                                         1516    ( 6.4%)
BOTH -> excluded                                                            335    ( 1.4%)
kept                                                                      23473
```

A CUT BY DATE ALONE WOULD HAVE COST 8,352 GAMES TO PROTECT AGAINST 335 (CHANGELOG-REGMC 0.3.0). The coordinator proposed the date cut, called it cheap without measuring it, measured it at 35% of the store, and was still proposing it when Will replaced it with the conjunction.

Twenty-five times more evidence was about to be thrown away than the change could possibly have affected.

AND THE CONJUNCTION IS EXACT, NOT AN APPROXIMATION — WILL'S ARGUMENT, THEN MEASURED. The obvious objection is an item acquired mid-battle, which a holder's own sheet would not show. Will: *"we would know if it acquires one because the opposing team sheet would contain it"* — under open team sheets every item on the field is declared by one of the two players, so scanning BOTH sheets catches a transferred item whoever ends up holding it. Measured over the old-rule window (CHANGELOG-REGMC 0.3.0): **335 games carry the item on a sheet, 335 name it anywhere in the record, and 0 name it outside a sheet.** There is no hidden case.

The cut is applied when the frozen M-C pool is built, and both sheets are scanned.

LEGALITY IS WHAT THE `TeamValidator` ACCEPTS — WILL, 2026-09-20

"lets use the showdown team validator again for reg mc".

Not `!isNonstandard` . Not a hand-rolled predicate. Not a learnset walk. The validator is what the live ladder runs, so it is the authority on what a legal set is.

THIS IS NOT A PREFERENCE, AND THE TWO DISAGREE IN THIS REGULATION. The strict filter drops one live ability that the validator accepts — see the trap above. When they disagree, the validator wins.

How the two fit together. The strict filter stays for BULK WALKS: it is right, and it exists because an unfiltered walk once invented five exceptions to a rule that has none. But its output is a CANDIDATE LIST, not the answer. Re-admit the `Future` entries the validator accepts, and print the re-admitted list on every run so it can never be silently empty.

For a fixture, validate before staging. A set the validator rejects is not a fixture, whatever a learnset table says. Reg M-B paid for this: a probe built its fixture from a raw learnset walk, staged an illegal set and went RED, and the red said nothing about the mechanic. **A `COULD-NOT-STAGE` verdict is always a claim about the fixture, never about the mechanic.**

For the stored games, the direction reverses. Every team in the store was already accepted by the live server, so a team in the pool is legal by construction. A predicate that rejects one is wrong about the predicate. Validate stored teams to MEASURE — how many would a given filter have wrongly rejected — and never to discard.

WHAT EXISTS TODAY, AND WHAT DOES NOT

THE FROZEN POOL IS CUT. `data/team-pool-frozen-regmc/` — the sample every M-C figure will be measured against, built to the same contract as Reg M-B's: the differential draws its teams LIVE from the store unless pinned, so two runs an hour apart otherwise ask different questions and neither is wrong. Its `FROZEN.md` states the predicate; `pool-receipt.json` digests the paths the cutter actually opened, not the canonical paths for that kind of file.

```
$ node engine/cut_regmc_pool.js          (stores read as tracked blobs at commit 2b25bd0f)
kept          24832 games, 49664 sides, 11608 distinct teams   digest 792daded918f
  bo3          23473 of 23808
  ladder       1359 of 33275
excluded      31888 not open-sheet (ladder only; the scope rule)
               363 the Eject Button conjunction: date < 2026-09-14 AND the item on either sheet
                  (335 bo3 + 28 ladder)
```

The conjunction was re-derived rather than inherited, and it held: 0 in-scope games name the item anywhere without it being on a sheet (CHANGELOG-REGMC 0.3.0). The raw ladder store names it far more often than it is declared — but that residue is CLOSED-sheet games, where the conjunction cannot be evaluated at all, and the figures are in `docs/_reports/2026-09-20-regmc-pool.md` rather than restated here. **There is no hidden case precisely BECAUSE the scope is open sheets**, which is worth knowing before anyone widens the scope later.

THE USAGE MODEL CHOMP READS IS PER REGULATION (0.23.0). `node engine/analyze.js --regulation regmc` writes `data/meta-usage-regmc.json` from the LIVE M-C stores with this pool's predicate (`engine/regmc_pool_predicate.js`, the one module both read) and the shared quality filter, and audits every entity in it for legality. It is live on purpose: it describes the meta for CHOMP and is not a measurement of MEDICHAM. CHOMP does not read it yet (`docs/_reports/2026-09-22-regmc-usage.md` §5).

The live stores keep growing — the collector appends hourly, so re-derive their counts rather than quoting any number written here.

Nothing is simulated yet. No M-C engine, no census, no roster, no gate. The checkout exists, the delta is derived, and that is the whole of it. *(Dated. Since then the engine, the gate and the census have been built: the census and its pin at 0.19.0, CHANGELOG-REGMC.md. Read the gate with `node engine/quarantine.js --regulation regmc`, not this sentence.)*

THE M-C POOL CARRIES CUSTOM-RULE ROOMS, AND THE M-B ANSWER DOES NOT TRANSFER — 0.6.0, 2026-09-21

A tournament room played under Showdown's custom rules is indexed under the **base format id**, so `search.json?format=` collects it like any other game and `engine/durable-ingest.js` cannot tell the difference — a format id does not encode custom rules. On the Reg M-B ladder that share is stated in `data/quality-filter.json:provenance.funnel.after_custom_ruleset` against `.collected` (the same file), and all of it is out of scope; `data/quality-filter.json` 1.6.0 excludes it (`docs/_reports/2026-09-21-custom-ruleset-filter.md`).

On Reg M-C the same instrument gives the opposite answer, so the rule is NOT copied across.

These counts are a LIVE DERIVATION over untracked store files, not a row in a `data/*.json` artifact, so they carry no artifact citation and are quoted as the readout that produced them:

```
$ node engine/scan_custom_rulesets.js --raw data/games.gen9championsvgc2026regmc.raw-logs.jsonl \
  --store data/games.gen9championsvgc2026regmc.jsonl.gz --out <scratch>
raw logs          1,412 records  (2026-09-09T20:52:08Z)
custom-rule infobox 57 (4.04%) in 2 distinct rule strings -- 41 Force Open Team Sheets, 16 Best of =
alters legality/pick 0
store             33,743 unique ids
JOINED            57 = 0.17% of the store
UNTESTABLE        32,331 store ids have no raw log on disk (95.82%) -- a FLOOR, not a census

$ (the same id set, joined against the frozen pool)
data/team-pool-frozen-regmc/games.bo3.jsonl  23,473 rows    0 custom-rule games
data/team-pool-frozen-regmc/games.ots.jsonl   1,359 rows    44 custom-rule games (3.24%)
  39 Force Open Team Sheets
   5 Best of = 3
```

`Force Open Team Sheets` **is the rule that MAKES a ladder game open-sheet, and open sheets are this line's scope.** Excluding those rows would delete the evidence the scope was chosen to collect. The `Best of = 3` handful is a different information regime sitting in the ladder half of the pool and is the only candidate for exclusion. **This is a judgement and it has not been taken** — the pool is frozen and cut, and nothing is changed here on the strength of five games.

AND THE M-C ANSWER IS MOSTLY UNASKED. The infobox lives in the RAW log, the M-C raw-log files stop on 2026-09-09, and the stores run to 2026-09-21 — so the scan reached about four percent of the store. That is not *"M-C is nearly clean"*; it is *"M-C has barely been asked"*. The scan prints that share on every run for exactly this reason. Full account: docs/_reports/2026-09-21-custom-ruleset-filter.md.

One thing found on the way, not fixed here, OPS's: `data/games.gen9championsvgc2026regmc.jsonl` on disk is a stale snapshot of 2026-09-09 while `data/games.gen9championsvgc2026regmc.jsonl.gz` beside it is current and far larger. `engine/quality.js` and `engine/quality.py` both prefer the PLAIN file when both exist — correct for the ladder store, which a local collector appends to, and **wrong here**, where the next-regulation collector produces the compressed one. Any reader of the M-C store today silently gets a fraction of it, which is a capability absent with everything reporting success.

Not started, in the order they unblock each other: a frozen M-C team pool cut from the store; the 41 new mechanics staged (CHANGELOG-REGMC 0.1.0); the census extended; the differential run against the M-C authority; the gate's clauses re-pointed. Each one is an ordinary piece of work — the hard part, knowing what actually changed, is done and is on this page.